Pricing Baku Homes From Scratch: A NumPy Decision Tree and Pegasos SVM on bina.az Listings

Surname1, Name1 Surname2, Name2 Surname3, Name3

MLE-AI-201 Machine Learning, AI Academy — Cohort II

***Abstract*—**We clean a scraped dump of 100,775 bina.az sale records into 57,422 unique Baku residential listings. We then frame two tasks: regression of log(price), and a premium/standard price-tier label split at the training-set median (209,000 AZN). Both models are implemented in NumPy. The first is a CART decision tree with Gini, entropy and MSE criteria, a vectorised threshold search and four stopping rules. The second is a soft-margin SVM trained by Pegasos sub-gradient descent on the hinge-loss primal, extended to an RBF kernel with random Fourier features. Hyperparameters are chosen on a validation split only. On the held-out test set our tree reaches RMSE 0.231 on log price ($R^2 = 0.869$, mean absolute percentage error 14.7% in AZN). scikit-learn's tree with the same settings scores 0.232. On the tier task our tree reaches F1 0.923 (sklearn 0.927), our linear SVM 0.909 (SGDClassifier 0.907) and our RFF-SVM 0.913 (kernel SVC 0.910). The from-scratch models match the library baselines within noise. Most remaining errors are luxury properties, house-heavy suburban districts and listings priced close to the tier threshold. Code: `github.com/TODO-team/TODO-repo` (tag `v1.0-final`).

***Index Terms*—**decision tree, support-vector machine, Pegasos, random Fourier features, real-estate price prediction.

I. Introduction

Online listing prices are noisy, heavy-tailed and dominated by location. Predicting them is a realistic test of whether a model has learned the market or just memorised the data. Our contributions: (i) a documented, leakage-free cleaning pipeline for the raw bina.az dump; (ii) a from-scratch CART tree whose splits match scikit-learn's exactly on a fully determined test example; (iii) a from-scratch Pegasos SVM [1] with visible margins and support vectors, plus a random-Fourier-feature (RFF) kernel extension [2]; (iv) a matched benchmark against scikit-learn [4] with cross-validated uncertainty, timing and an error analysis. `python -m src.run_all` regenerates every number, table and figure in this report.

II. Background

A classification tree chooses, at each node, the split $x_j \le t$ that maximises the information gain $\mathrm{IG} = I(\text{parent}) - \sum_c \frac{N_c}{N} I(c)$, where I is the Gini impurity $1 - \sum_k p_k^2$ or the entropy $-\sum_k p_k \log_2 p_k$. A regression tree uses the variance $I_{\text{reg}} = \frac{1}{N}\sum_i (y_i - \bar{y})^2$ instead [3]. The soft-margin linear SVM minimises the regularised hinge loss

$$J(\mathbf{w}, b) = \tfrac{\lambda}{2}\|\mathbf{w}\|^2 + \tfrac{1}{n}\sum_{i=1}^{n} \max\big(0,\, 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)\big), \quad (1)$$

which Pegasos solves by stochastic sub-gradient steps with rate $\eta_t = 1/(\lambda t)$, with no QP or SMO solver [1]. This objective equals the textbook C-SVM with $C = 1/(\lambda n)$.

III. Data and Preprocessing

A. Dataset and cleaning

The Kaggle dump (`house_sale.csv`, version 1, scraped Oct–Nov 2024) has 100,775 rows and 51 columns. Most columns are strings with Azerbaijani names and units. The cleaning steps, in order, are logged by `data_prep.clean`:

- *Currency:* every price is AZN (checked), so no conversion is needed.
- *Scope:* we keep residential categories (new build, old build, house/villa) in Baku. Land, offices, garages and commercial objects are dropped because they form different markets, and land is priced per *sot*.
- *Duplicates:* the same listing is re-scraped on many days. We keep the newest scrape per URL, which removes 33,054 rows, and then drop 709 re-posts with identical price, area, rooms, floor, coordinates and category.
- *Type coercion:* `Sahə` ("145 m²")→ area; `Torpaq sahəsi` ("1.3 sot")→ land; `Mərtəbə` ("7 / 9")→ floor and total floors; yes/no columns→0/1. The district (*rayon*) and metro proximity are parsed from `extra_info`.
- *Outliers:* we use fixed domain rules, not data quantiles, so no statistic is learned from test rows. The rules drop rows with price outside $[10^4, 5{\cdot}10^6]$ AZN, area outside $[15, 1500]$ m², price per m² outside $[150, 15000]$ (typos or rent ads), land above 200 sot, more than 15 rooms, impossible floors, or coordinates outside Baku. Together they remove 133 rows.
- *Missing values:* median imputation using *training* medians, with a missing-indicator column for rooms, floor and repair. Missing land area means a flat, so it is set to 0.

B. Leakage

`unit_price` (AZN per m²) is price divided by area, and `total_price` equals `price` in 99.99% of rows. Both are arithmetic functions of the target and are dropped. Including either one would let the tree reconstruct the price exactly. We also drop identifiers and personal fields (`owner_name`, `shop_name`, `address`, ids and URLs), the free-text `description`, which often restates the price, scrape timestamps, and `views`, which accumulates *after* listing. `run_all` asserts that no leakage column survives into the feature names.

Table 1. Dataset summary after cleaning.

Property	Value
Raw rows (scraped records)	100,775
Rows after cleaning	57,422
Features (after one-hot)	99
Train / Val / Test	40,196 / 8,613 / 8,613
Tier threshold (train median)	209,000 AZN
Premium share train / val / test (%)	49.8 / 49.9 / 49.8

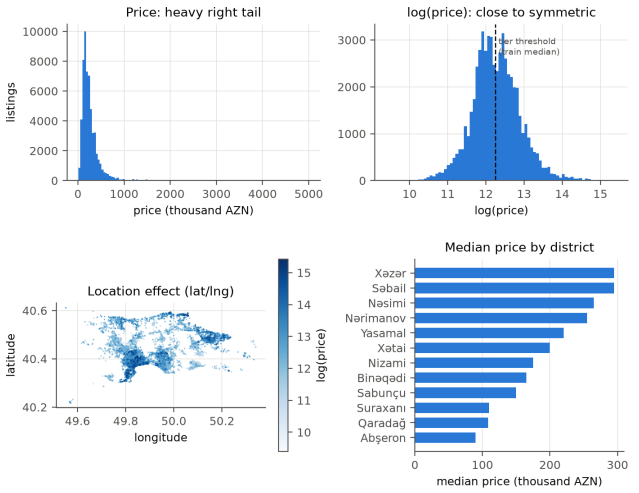


Figure 1. Price is heavy-tailed; log(price) is close to symmetric (top). Location matters (bottom): the seaside villa district Khazar and the central Sabail, Nasimi and Narimanov have the highest median prices, while the outer Absheron, Qaradagh and Surakhani have the lowest.

C. Features, tasks and split

The 99 features are: numeric attributes (area, rooms, floor, total floors, land, lat/lng); engineered features (floor ratio, top and ground floor flags, area per room, haversine distance to the city centre); binary amenities (repair, bill of sale, mortgage, agent, residential complex, VIP/featured); and one-hot category, district and neighbourhood columns. Neighbourhoods with fewer than 100 training listings are pooled into "other". We chose one-hot over ordinal codes because a linear SVM would otherwise read a fake ordering into nominal values. The split is 70/15/15 and stratified on price deciles, which balances the tier label across splits (Table 1). The tier threshold is the median of the *training* prices only. Task A is evaluated with RMSE, MAE and R^2 on log price and on AZN. Task B is evaluated with precision, recall, F1, ROC-AUC, PR-AUC and the confusion matrix.

IV. Method

A. Decision tree (from scratch)

`DecisionTree(task, criterion, max_depth, min_samples_split, min_samples_leaf, min_impurity_decrease)` mirrors sklearn's `fit/predict/predict_proba`. For every node and feature we sort the samples once. Prefix sums of class counts (classification) or of y and y^2 (regression) then give the impurity of *every* candidate split in one vectorised pass. Candidates lie only between two distinct values, the threshold is their midpoint, and children smaller than `min_samples_leaf` are excluded. The growth is recursive and stops at `max_depth`, below `min_samples_split`, at a pure node, or when the weighted decrease $\frac{N_t}{N}$IG falls below `min_impurity_decrease` (sklearn's definition). A leaf predicts the majority class (with `predict_proba` from leaf class frequencies) or the mean. Feature importances are the normalised sums of $N_t \cdot$IG. Two numerical details turned out to matter. First, ties go to the lowest feature index, whereas sklearn permutes features at random. Second, when two neighbouring values are adjacent doubles, their midpoint rounds onto the upper value, so we fall back to the lower one. Our real data triggered this bug and it is now covered by a test.

B. Support-vector machine (from scratch)

`PegasosSVM` runs mini-batch Pegasos (batch 32; batch 1 and full batch are also supported). Each step applies $\mathbf{w} \leftarrow (1-\eta_t\lambda)\mathbf{w} + \frac{\eta_t}{|B|}\sum_{i\in A} y_i\mathbf{x}_i$ over the margin violators $A = \{i : y_i\mathbf{w}^\top\mathbf{x}_i < 1\}$. The bias is folded in as a constant feature, as the brief suggests. Optional steps are the projection onto $\|\mathbf{w}\| \le 1/\sqrt{\lambda}$ and averaging of the second-half iterates. The averaging is the variant covered by the Pegasos guarantee, and in practice it removes the noise of the $1/(\lambda t)$ schedule. The model exposes λ, the epochs, the batch size and the learning-rate schedule ($1/(\lambda t)$, $\eta_0/\sqrt{t}$ or constant). It records J after every epoch and reports $\|\mathbf{w}\|$, the margin $2/\|\mathbf{w}\|$ and the support vectors ($y_i f(\mathbf{x}_i) \le 1$). Features are standardised with training statistics.

Non-linearity: option (ii), random Fourier features. We map $\phi(\mathbf{x}) = \sqrt{2/D}\cos(W\mathbf{x}+\mathbf{c})$ with $W_{ij} \sim \mathcal{N}(0, 2\gamma)$ and $c_j \sim U[0, 2\pi)$, so that $\phi(\mathbf{x})^\top\phi(\mathbf{z}) \approx e^{-\gamma\|\mathbf{x}-\mathbf{z}\|^2}$ [2], and run the same linear Pegasos on $\phi(X)$ with $D = 1000$. We chose RFF over kernelised Pegasos because, with ∼40k training rows, kernel Pegasos needs $O(n^2)$ kernel evaluations, while RFF costs $O(nD)$ and reuses the tested linear solver unchanged.

C. Tests

The pytest suite (32 tests) covers the following. Our tree reproduces sklearn's exact split list (feature, threshold, depth) and `predict_proba` on a 12-point example for Gini, entropy and MSE. Each of the four hyperparameters changes the tree, and feature importances match sklearn's. Pegasos decreases J, agrees with `SVC` ($C = 1/(\lambda n)$) on more than 97% of a toy split with weight cosine above 0.99, and RFF approximates the RBF kernel and solves XOR. All metrics equal sklearn's.

V. Experimental Setup

All tuning uses the validation split, and the test split is touched once at the end. For the tree we swept `max_depth`×`min_samples_leaf` for Task A and criterion×`max_depth`, then `min_samples_leaf`, for Task B. For the SVM we swept $\lambda \in [10^{-6}, 10^{-1}]$, compared schedules and epochs, and ran a $\gamma \times \lambda$ grid for RFF. For the SVM we pick the *most regularised* setting whose validation F1 is within 0.002 of the best. Smaller differences are below the noise level, and very small λ

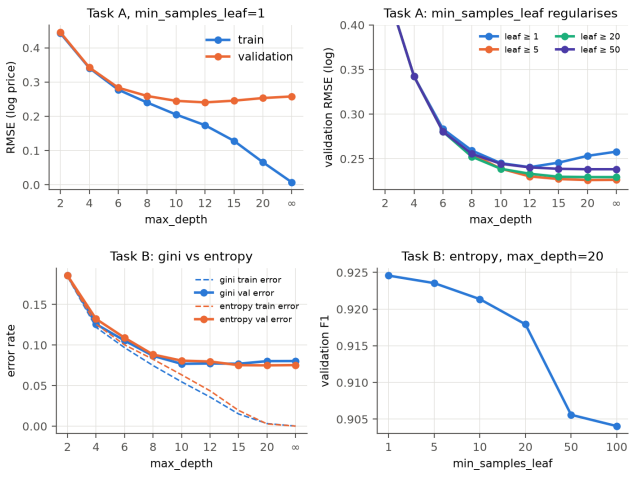


Figure 2. Our tree's validation curves. Top: Task A train vs validation RMSE (left) and the effect of `min_samples_leaf` (right). Bottom: Gini vs entropy over depth, and the leaf-size sweep.

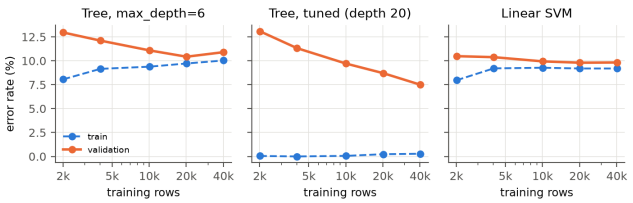


Figure 3. Learning curves for Task B: shallow tree, tuned deep tree and linear SVM (train vs validation error).

Table 2. Linear Pegasos SVM: λ sweep (validation).

λ	val F1	val AUC	$\|w\|$	$2/\|w\|$	SV %	J
1e-01	0.871	0.954	1.16	1.731	46.0	0.393
1e-02	0.899	0.964	2.27	0.883	31.3	0.288
1e-03	0.902	0.965	3.39	0.591	25.7	0.258
1e-04	0.903	0.965	5.91	0.338	19.8	0.265
1e-05	0.903	0.964	25.32	0.079	11.2	0.627
1e-06	0.903	0.964	221.83	0.009	9.4	4.710

had not converged in 30 epochs (Table 2). The final configurations are: tree A depth 20, leaf $\geq$5; tree B entropy, depth 20, leaf $\geq$1; linear SVM $\lambda = 0.001$; RFF $\gamma = 0.003$, $\lambda = 1e-05$. The baselines use the *same* hyperparameters: `DecisionTreeRegressor/Classifier`, and `Pipeline(StandardScaler, SGDClassifier(hinge, alpha=`λ`))` as the matched linear SVM. `SVC` (linear and RBF, $C = 1/(\lambda n)$, same γ) is fit on a stratified 10k subsample because its training time grows super-linearly (Fig. 6). Uncertainty is reported in two ways: a bootstrap std on the test set (200 resamples), and 5-fold CV mean$\pm$std on train+val, with scaling refit inside each fold. The imputation medians and one-hot vocabularies are fit once on the training split and reused in the CV folds. This is a mild optimism for the folds drawn from the validation rows, and it does not affect the test numbers.

VI. Results

A. Tuning and the bias–variance story

Fig. 2 shows the classic picture. Training error keeps falling with depth, but with leaves of size 1 the validation RMSE bottoms out at depth 12 and then rises as the tree memorises individual listings. A minimum leaf size of 5–20 removes most of that variance. Gini and entropy are within noise of each other (best val F1 0.9227 vs 0.9246). The learning curves (Fig. 3) show both failure modes. The tuned deep tree has near-zero training error and a large gap that shrinks as data is added, which is high variance, so more data still helps. The depth-6 tree and the linear SVM have training error close to validation error, which is high bias: more data does not help them, and only more capacity would.

For the SVM (Table 2, Fig. 4), shrinking λ grows $\|\mathbf{w}\|$, narrows the margin and reduces the number of support vectors. Validation F1 saturates at $\lambda \leq 10^{-3}$. Below that, 30 epochs no longer reach the optimum (J rises again in Table 2) because the $1/(\lambda t)$ steps are huge early on. At the chosen λ the averaged, last-iterate and constant-rate runs all converge to $J \approx 0.258$ within 20 epochs. The full-batch run takes only one step per epoch and is still far from the optimum after 50. More epochs widen the margin towards its converged value, while F1 barely moves after two epochs. In the 2-D illustration (Fig. 5) one can see the margin band and the support vectors directly.

B. Benchmark against scikit-learn

Tables 3–4 and the combined Table 5 hold the headline results. On Task A our tree and sklearn's are within one bootstrap std (0.231 vs 0.232 RMSE on log price), and both are far better than predicting the mean (0.636). In AZN the MAE is about 46.1k AZN. On Task B every model lands in a narrow band: our tree 0.923 vs sklearn 0.927; our linear SVM 0.909 vs SGD 0.907 and SVC 0.904; our RFF-SVM 0.913 vs RBF-SVC 0.910. Our final linear SVM has $\|\mathbf{w}\| = 3.386$, margin $2/\|\mathbf{w}\| = 0.591$, and 25.7% of the training points are support vectors.

Structure (Table 6). With matched hyperparameters the two trees choose the same root split (area ≤ 100.25 m^2 for A) and share 6–7 of the first 7 splits. They agree on 93.6% (A) and 98.2% (B) of test predictions. The remaining gaps come from three sources: (a) sklearn stores X as float32, so thresholds are rounded and nearly-equal values can merge; (b) ties are broken by a random feature permutation in sklearn and by the lowest index in ours; (c) after an early disagreement the subtrees diverge.

Speed (Fig. 6). Our tree is written in Python over NumPy and fits in 17.3 s vs 0.70 s for sklearn's Cython (Task A), 25–45$\times$ slower across our runs, because every node pays Python overhead on top of the sort. Both trees grow slightly faster than linearly in n. Wall-clock times on a shared laptop are noisy. Our Pegasos fits in 5.0 s and is linear in n. Kernel SVC is already slower on a 10k subsample (7.0 s) and its cost grows super-linearly, which is why the RFF route is the practical way to get a kernel at this size.

C. Error analysis

What the models use. The tree's impurity-decrease importances (Fig. 7) are dominated by area (0.59) and location (distance to the centre 0.23, then longitude and latitude), followed by land area and category. Our importances and

Table 3. Task A (log price) on the test set. RMSE ± bootstrap std; CV = 5-fold mean ± std on train+val.

Model	RMSE log	MAE log	R^2 log	RMSE AZN	MAE AZN	R^2 AZN	CV RMSE log	fit s
Baseline: predict train mean	0.636 ± 0.006	0.490	-0.000	264.7k	130.1k	-0.046	n/a	0.00
Ours: DecisionTree (MSE)	0.231 ± 0.004	0.142	0.869	134.7k	46.1k	0.729	0.234 ± 0.003	17.31
sklearn DecisionTreeRegressor	0.232 ± 0.004	0.143	0.867	135.0k	46.3k	0.728	0.233 ± 0.002	0.70

Table 4. Task B (price tier) on the test set. F1 ± bootstrap std; CV F1 = 5-fold mean ± std on train+val. SVC is fit on a stratified 10k subsample of the training split.

Model	Prec.	Rec.	F1 (test)	ROC-AUC	PR-AUC	CV F1	fit s	pred. s
Ours: DecisionTree (entropy)	0.922	0.924	0.923 ± 0.003	0.926	0.894	0.923 ± 0.003	9.83	0.031
sklearn DecisionTreeClassifier	0.927	0.927	0.927 ± 0.003	0.930	0.900	0.923 ± 0.003	1.31	0.006
Ours: Pegasos linear SVM	0.910	0.908	0.909 ± 0.003	0.968	0.965	0.906 ± 0.003	4.98	0.201
Ours: RFF-Pegasos SVM	0.912	0.913	0.913 ± 0.003	0.974	0.973	0.909 ± 0.002	9.10	0.316
sklearn SGDClassifier (hinge)	0.906	0.909	0.907 ± 0.003	0.967	0.964	0.905 ± 0.003	1.83	0.010
sklearn SVC linear (10k subsample)	0.906	0.903	0.904 ± 0.003	0.966	0.963	0.905 ± 0.003	6.29	2.994
sklearn SVC RBF (10k subsample)	0.911	0.908	0.910 ± 0.003	0.973	0.973	0.908 ± 0.003	7.01	4.800

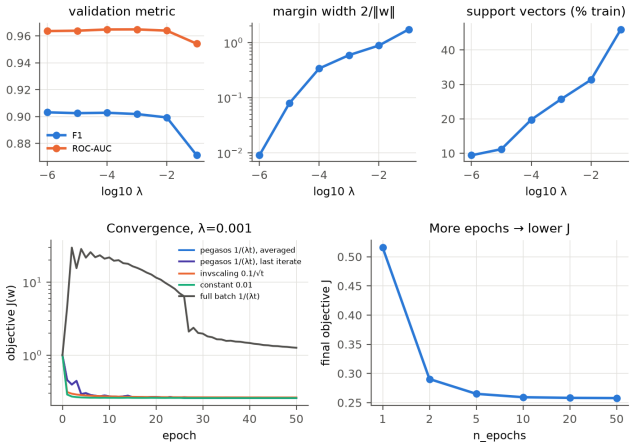


Figure 4. SVM: λ trades margin width against fit (top). The objective J decreases and converges, compared across schedules and epochs (bottom).

Table 5. All models on both tasks (test set; CV on train+val).

Model	A: RMSE log	A: R^2 log	A: CV RMSE	B: F1	B: AUC	B: CV F1
Baseline: predict train mean	0.636	-0.000	n/a	—	—	—
Ours: DecisionTree (MSE)	0.231	0.869	0.234 ± 0.003	—	—	—
sklearn DecisionTreeRegressor	0.232	0.867	0.233 ± 0.002	—	—	—
Ours: DecisionTree (entropy)	—	—	—	0.923	0.926	0.923 ± 0.003
sklearn DecisionTreeClassifier	—	—	—	0.927	0.930	0.923 ± 0.003
Ours: Pegasos linear SVM	—	—	—	0.909	0.968	0.906 ± 0.003
Ours: RFF-Pegasos SVM	—	—	—	0.913	0.974	0.909 ± 0.002
sklearn SGDClassifier (hinge)	—	—	—	0.907	0.967	0.905 ± 0.003
sklearn SVC linear (10k subsample)	—	—	—	0.904	0.966	0.905 ± 0.003
sklearn SVC RBF (10k subsample)	—	—	—	0.910	0.973	0.908 ± 0.003

sklearn's are almost identical. The SVM weights tell the same story with signs: area, rooms and area per room push towards "premium", while distance from the centre and outer settlements such as Binə push towards "standard".

Where the models fail. (i) *Luxury and bargain tails:* the tree shrinks towards the middle. It over-predicts the cheapest decile and under-predicts the most expensive one (Fig. 8, left), because a leaf mean cannot extrapolate. The largest AZN errors (Table 7) are 700–1200 m^2 villas listed at 3–5M AZN, plus one large new-build flat. (ii) *Houses and the districts dominated by them:* the MAE on log price for

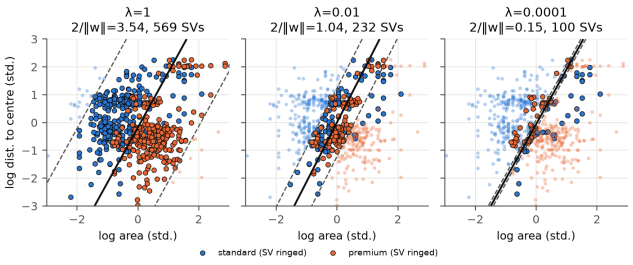


Figure 5. A margin you can see: our SVM on two standardised features. Solid line: $f = 0$; dashed: $f = \pm 1$; ringed points: support vectors.

Table 6. Our tree vs sklearn's with matched hyperparameters.

	depth (ours/sk)	leaves (ours/sk)	first 7 splits equal	same test pred.
Task A	20 / 20	5324 / 5337	6/7	93.6%
Task B	20 / 20	2231 / 2239	7/7	98.2%

houses is about 2.6× that for flats (Fig. 8, right), because land, condition and plot shape vary far more and are only partly captured by our features. Per district (12 districts with at least 10 test rows), the test MAE correlates strongly with the share of houses ($r = 0.85$) and more weakly with the number of training listings ($r = -0.55$). The smaller districts are also the suburban, house-heavy ones, so the two effects overlap. House share is the better explanation: Khazar, almost entirely villas, is the worst district despite 2.7k training listings. With only 12 points this is suggestive, not conclusive. (iii) Flats in the dense central districts have the smallest errors (MAE ≈0.09–0.12 on log price, i.e. roughly 10%). (iv) *Tier errors near the threshold:* 77% of the SVM's tier errors are listings within about 20% of the 209,000 AZN threshold (Fig. 9). Many of these are irreducible, because asking prices of near-identical flats differ by more than that. Near the threshold the deep tree errs less than both SVMs, since it can carve out local neighbourhoods. Far from the threshold the smoother SVMs make fewer mistakes than the tree, whose leaves sometimes hold a few mislabelled out-

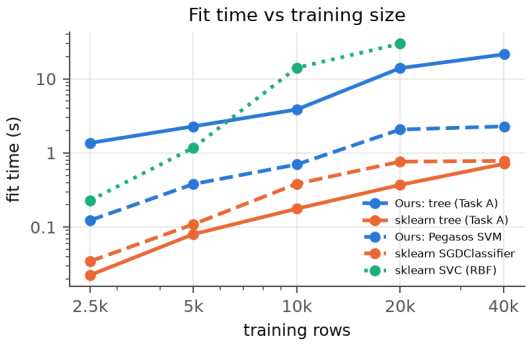


Figure 6. Fit time vs training size (log–log).

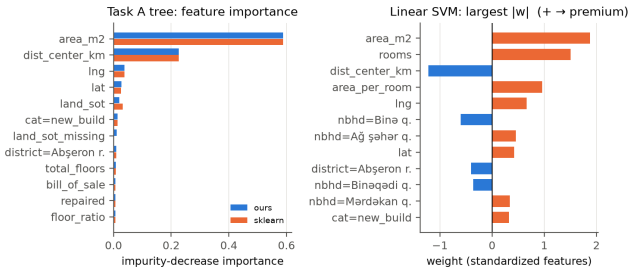


Figure 7. Left: impurity-decrease importances (Task A), ours vs sklearn. Right: largest linear-SVM weights (Task B; positive pushes towards premium).

liers. This is consistent with the SVMs' higher ROC-AUC despite a lower F1.

VII. Discussion and Limitations

- *Asking, not transaction, prices.* The target is the listed price. Agents' over-pricing and stale listings add noise that no feature can explain.
- *One snapshot.* The data covers five weeks in autumn 2024. A random split mixes the same buildings into train and test, and a time- or building-grouped split would be harder. We did not model time.
- *Unused text.* `description` probably holds strong signal (renovation quality, view), but it sometimes restates the price, so we excluded it rather than risk leakage.
- *Speed.* Our tree is 25–45× slower than sklearn's Cython. This is acceptable here, but a histogram-based split search would be needed for much larger data.
- *SVC on a subsample.* The kernel-SVC baseline sees 10k rows, not 40k. That favours our models slightly, and we state it in every table.
- *Residual duplicates.* Three identical rows in Table 7 (same district, area and price) suggest that re-posts with slightly different coordinates survive our exact-match de-duplication. A fuzzy match (e.g. same price and area within 200 m) would be stricter.
- *Selection rule.* Choosing the most regularised SVM within 0.002 F1 is a judgement call. The validation-best $\lambda = 10^{-6}$ would give the same F1 within noise, but with an unconverged objective.

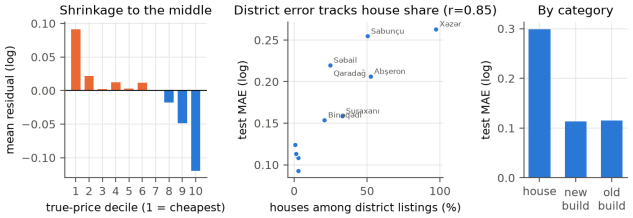


Figure 8. Task A test errors of our tree: mean residual by true-price decile (left), district MAE vs the share of houses in the district (middle), and MAE by category (right).

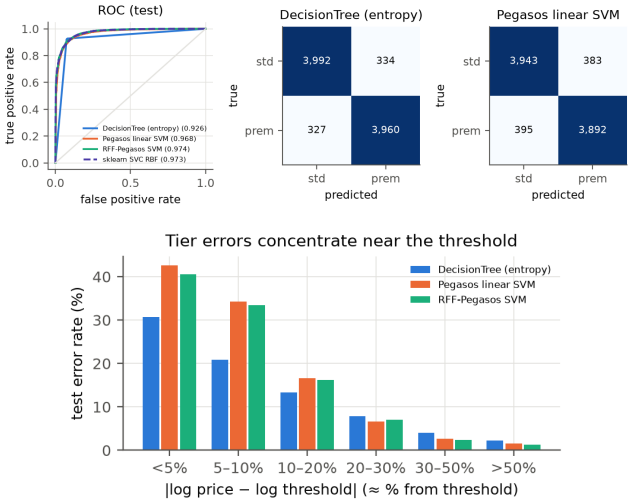


Figure 9. Task B: ROC curves and confusion matrices (test). Error rate vs distance of the true price from the tier threshold.

VIII. Conclusion

From-scratch implementations of CART and Pegasos, checked against sklearn on toy and real data, reach the same accuracy as the library versions on a real, messy dataset once leakage and duplicates are handled. Location and area explain most of the price. The remaining error sits in the tails and close to the tier threshold. Next steps are a grouped/temporal split, text features from descriptions with price mentions masked, and ensembles of our own trees.

Tools and Acknowledgements

AI assistance: an AI coding assistant (Claude, Anthropic) was used to draft the code, tests, this report and the slides. All results were produced by running the code in the repository, and the team reviewed and can explain every component (see `contribution_report.md`). *Data:* bina.az listings via Kaggle (`sehriyarmemmedli/binaaz-sale-project`). Personal fields were dropped and are not redistributed. *Libraries:* NumPy, pandas and Matplotlib, with scikit-learn used for baselines and tests only.

References

[1] S. Shalev-Shwartz, Y. Singer, N. Srebro, and A. Cotter, "Pegasos: primal estimated sub-gradient solver for SVM," *Mathematical Programming*, vol. 127, no. 1, pp. 3–30, 2011.

Table 7. Largest absolute AZN errors of our Task A tree (test).

category	district	m^2	true	pred.
house	Nerimanov r.	1000	5,000k	1,562k
house	Sabuncu r.	1200	4,500k	1,301k
house	Sebail r.	1000	3,500k	1,214k
house	Xezer r.	1000	4,000k	2,105k
house	Xezer r.	1000	4,000k	2,105k
house	Xezer r.	1000	4,000k	2,105k
new build	Nesimi r.	237	3,094k	1,232k
house	Sebail r.	700	3,000k	1,214k

[2] A. Rahimi and B. Recht, “Random features for large-scale kernel machines,” in *Proc. NeurIPS*, 2007, pp. 1177–1184.

[3] L. Breiman, J. Friedman, R. Olshen, and C. Stone, *Classification and Regression Trees*. Wadsworth, 1984.

[4] F. Pedregosa *et al.*, “Scikit-learn: machine learning in Python,” *JMLR*, vol. 12, pp. 2825–2830, 2011.